

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	SOAP Web services				
NÚMERO DE LA PRÁCTICA:	07	AÑO LECTIVO:	2026	NRO. SEMESTRE:	A
FECHA DE PRESENTACIÓN:	01/06/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(S): - Bedregal Perez Daniel - Jara Mamani Mariel Alisson - Mestas Zegarra Christian Raul - Noa Camino Yenaro Joel - Sequeiros Condori Luis Gustavo				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Mg. Maribel Molina Barriga					

RESULTADOS Y PRUEBAS
ENLACE A GITHUB https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/l7 EJERCICIOS RESUELTOS POR EL DOCENTE Se replicó el servicio SOAP de suma para validar publicación y consumo mediante JAX-WS. Servicio SOAP de Suma (Calculadora) <pre>@WebMethod public int sumar(int a, int b) { return a + b; } public static void main(String[] args) { Endpoint.publish("http://localhost:8080/calculadora", new CalculadoraSOAP()); System.out.println("Servicio SOAP activo"); } public static void main(String[] args) throws Exception { URL url = new URL("http://localhost:8080/calculadora?wsdl"); QName qname = new QName("http://lab7.docente/", "CalculadoraSOAPService"); Service service = Service.create(url, qname); CalculadoraSOAP calc = service.getPort(CalculadoraSOAP.class); System.out.println(calc.sumar(10, 20)); }</pre> EJERCICIOS/PROBLEMAS PROPUESTOS Ejercicio 1: Conversor de Temperatura Se implementó el servicio SOAP con las operaciones cToF y fToC, junto con el publicador y el cliente Java.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```
@WebMethod
double cToF(double c);

@WebMethod
double fToC(double f);
```

```
@Override
public double cToF(double c) {
    return (c * 9.0 / 5.0) + 32.0;
}

@Override
public double fToC(double f) {
    return (f - 32.0) * 5.0 / 9.0;
}
```

```
public static void main(String[] args) {
    Endpoint.publish("http://localhost:8080/conversor", new ConversorSOAP());
}
```

```
public static void main(String[] args) throws Exception {
    URL wsdlUrl = new URL("http://localhost:8080/conversor?wsdl");
    QName serviceName = new QName("http://l7.e1.conversor/", "ConversorSOAPService");
    Service service = Service.create(wsdlUrl, serviceName);
    ConversorAPI api = service.getPort(ConversorAPI.class);

    System.out.println("30C -> " + api.cToF(30));
    System.out.println("86F -> " + api.fToC(86));
}
```

Ejecucion del servicio

```
cd l7/src/e1
javac -d . $(find conversor -name "*.java")
java -cp . lab7.e1.conversor.PublishService
```

Verificacion del WSDL

Ruta: <http://localhost:8080/conversor?wsdl>
Resultado: se genero el contrato WSDL con operaciones cToF y fToC.

```
<definitions name="ConversorSOAPService">
  <portType name="ConversorAPI">
    <operation name="cToF" />
    <operation name="fToC" />
  </portType>
  <service name="ConversorSOAPService">
    <port name="ConversorSOAPPort" />
  </service>
</definitions>
```

Pruebas del endpoint (curl)

Ruta: <http://localhost:8080/conversor>
Metodo: POST (SOAP 1.1)
Operacion: cToF
Entrada: 30
Resultado: 86.0

```
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:cToFResponse xmlns:ns2="http://lab7.e1.conversor/">
      <return>86.0</return>
    </ns2:cToFResponse>
  </S:Body>
</S:Envelope>
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

```

</S:Body>
</S:Envelope>
Ruta: http://localhost:8080/conversor
Metodo: POST (SOAP 1.1)
Operacion: fToC
Entrada: 86
Resultado: 30.0
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:fToCResponse xmlns:ns2="http://lab7.e1.conversor/">
      <return>30.0</return>
    </ns2:fToCResponse>
  </S:Body>
</S:Envelope>

```

Ejercicio 1 (Adicional): Servicio SOAP de Ventas en Linea

Se diseño un servicio basico de ventas con listado, compra, registro, actualizacion y eliminacion logica de productos.

```

private String name;
private int cantidad;
private double costo;

private static final List<Item> ITEMS = new ArrayList<>(Arrays.asList(
    new Item("Gaseosa", 15, 5.2),
    new Item("Galletas", 10, 1.6),
    new Item("Celular", 12, 900.0)));

public Item() {
}

public Item(String name, int cantidad, double costo) {
    this.name = name;
    this.cantidad = cantidad;
    this.costo = costo;
}

public static List<Item> getItems() {
    return ITEMS;
}

public static boolean addItem(Item item) {
    if (item.name == null || item.name.isEmpty() || item.cantidad <= 0 || item.costo <= 0) {
        return false;
    }
    ITEMS.add(item);
    return true;
}

public static String buyItem(String name, int cantidad) {
    if (cantidad <= 0) {
        return "Invalid Quantity";
    }
    for (Item item : ITEMS) {
        if (item.name.equalsIgnoreCase(name)) {
            if (item.cantidad - cantidad < 0) {
                return "Not enough items";
            }
            item.cantidad -= cantidad;
            return "Producto: " + item.name + " Cantidad: " + cantidad
                + " Total: " + String.format(java.util.Locale.US, "%.2f", (cantidad * item.costo));
        }
    }
    return "Item no match";
}

public static boolean setItem(String name, int cantidad, double costo) {
    if (name == null || name.isEmpty() || cantidad <= 0 || costo <= 0) {
        return false;
    }
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```

for (Item item : ITEMS) {
    if (item.name.equalsIgnoreCase(name)) {
        item.cantidad = cantidad;
        item.costo = costo;
        return true;
    }
}
return false;
}

public static boolean deleteItem(String name) {
    if (name == null || name.isEmpty()) {
        return false;
    }
    for (Item item : ITEMS) {
        if (item.name.equalsIgnoreCase(name)) {
            item.cantidad = 0;
            return true;
        }
    }
    return false;
}

public String getNombre() {
    return name;
}

public void setNombre(String name) {
    this.name = name;
}

public int getCantidad() {
    return cantidad;
}

public void setCantidad(int cantidad) {
    this.cantidad = cantidad;
}

public double getCosto() {
    return costo;
}

public void setCosto(double costo) {
    this.costo = costo;
}

```

```

@WebMethod
List<Item> getItems();

@WebMethod
String buyItem(String name, int cantidad);

@WebMethod
boolean addItem(Item item);

@WebMethod
boolean setItem(String name, int cantidad, double precio);

@WebMethod
boolean deleteItem(String name);

```

```

@Override
public List<Item> getItems() {
    return Item.getItems();
}

@Override
public String buyItem(String name, int cantidad) {
    return Item.buyItem(name, cantidad);
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

```

}

@Override
public boolean addItem(Item item) {
    return Item.addItem(item);
}

@Override
public boolean setItem(String name, int cantidad, double precio) {
    return Item.setItem(name, cantidad, precio);
}

@Override
public boolean deleteItem(String name) {
    return Item.deleteItem(name);
}

public static void main(String[] args) {
    Endpoint.publish("http://localhost:1516/WS/Store", new SOAPImpl());
}

public static void main(String[] args) throws Exception {
    URL wsdlUrl = new URL("http://localhost:1516/WS/Store?wsdl");
    QName serviceName = new QName("http://l7.e1.store/", "StoreSOAPService");
    Service service = Service.create(wsdlUrl, serviceName);
    SOAPAPI store = service.getPort(SOAPI.class);

    List<Item> items = store.getItems();
    System.out.println("Lista de productos: " + Arrays.toString(items.toArray()));

    store.addItem(new Item("Chicle", 20, 1.4));
    System.out.println("Lista actualizada: " + Arrays.toString(store.getItems().toArray()));

    String recibo = store.buyItem("Galletas", 2);
    System.out.println("Recibo: " + recibo);

    store.setItem("Galletas", 18, 2.4);
    System.out.println("Lista final: " + Arrays.toString(store.getItems().toArray()));
}

```

Ejercicio 2: Cliente SOAP con Python

Se implemento un cliente interactivo con zeep para consumir el servicio SOAP de ventas en línea.

```

# pyrefly: ignore [missing-import]
from zeep import Client

def main():
    wsdl_url = "http://localhost:1516/WS/Store?wsdl"
    print(f"Conectando al servicio SOAP en: {wsdl_url}...")
    try:
        client = Client(wsdl_url)
        print("¡Conexión establecida con éxito!\n")
    except Exception as e:
        print(f"Error al conectar al servicio SOAP: {e}")
        return

    while True:
        print("=====")
        print("          MENÚ DE OPERACIONES SOAP          ")
        print("=====")
        print("1. Listar todos los productos (getItems)")
        print("2. Comprar un producto (buyItem)")
        print("3. Registrar nuevo producto (addItem)")
        print("4. Actualizar stock y precio (setItem)")
        print("5. Eliminar producto (deleteItem)")
        print("6. Salir")
        print("=====")

```



```
opcion = input("Seleccione una opción (1-6): ").strip()

if opcion == "1":
    print("\n--- LISTA DE PRODUCTOS ---")
    try:
        items = client.service.getItems()
        if not items:
            print("No hay productos disponibles.")
        else:
            for item in items:
                print(f"- Producto: {item.nombre:15} | Stock: {item.cantidad:5} | Costo: S/. {item.costo:.2f}")
    except Exception as e:
        print(f"Error al listar productos: {e}")
    print()

elif opcion == "2":
    print("\n--- COMPRAR PRODUCTO ---")
    name = input("Nombre del producto: ").strip()
    if not name:
        print("Nombre inválido.")
        continue
    try:
        cantidad = int(input("Cantidad a comprar: "))
        resultado = client.service.buyItem(name, cantidad)
        print(f"Resultado: {resultado}")
    except ValueError:
        print("Por favor, ingrese una cantidad numérica válida.")
    except Exception as e:
        print(f"Error al realizar la compra: {e}")
    print()

elif opcion == "3":
    print("\n--- REGISTRAR NUEVO PRODUCTO ---")
    name = input("Nombre del producto: ").strip()
    if not name:
        print("Nombre inválido.")
        continue
    try:
        cantidad = int(input("Cantidad inicial: "))
        costo = float(input("Costo/Precio: "))
        nuevo_item = {"nombre": name, "cantidad": cantidad, "costo": costo}
        bool = client.service.addItem(nuevo_item)
        if(bool):
            print(f"Producto '{name}' registrado exitosamente.")
        else:
            print(f'Hubo un problema y el Producto {name} no se pudo registrar')
    except ValueError:
        print("Por favor, ingrese valores numéricos válidos.")
    except Exception as e:
        print(f"Error al registrar producto: {e}")
    print()

elif opcion == "4":
    print("\n--- ACTUALIZAR PRECIO Y STOCK ---")
    name = input("Nombre del producto a actualizar: ").strip()
    if not name:
        print("Nombre inválido.")
        continue
    try:
        cantidad = int(input("Nueva cantidad (stock): "))
        costo = float(input("Nuevo costo/precio: "))
        bool = client.service.setItem(name, cantidad, costo)
        if(bool):
            print(f"Producto '{name}' actualizado exitosamente.")
        else:
            print(f'Hubo un problema y el Producto {name} no se pudo actualizar')
    except ValueError:
        print("Por favor, ingrese valores numéricos válidos.")
    except Exception as e:
        print(f"Error al actualizar producto: {e}")
    print()
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

```

elif opcion == "5":
    print("\n--- ELIMINAR PRODUCTO (Baja lógica) ---")
    name = input("Nombre del producto a eliminar: ").strip()
    if not name:
        print("Nombre inválido.")
        continue
    try:
        bool = client.service.deleteItem(name)
        if(bool):
            print(f"Producto '{name}' eliminado lógicamente (stock establecido en 0).")
        else:
            print(f'Hubo un problema y el Producto {name} no se pudo eliminar')
    except Exception as e:
        print(f"Error al eliminar producto: {e}")
    print()

elif opcion == "6":
    print("\nSaliendo del cliente interactivo. ¡Adiós!")
    break
else:
    print("\nOpción no válida. Intente nuevamente.\n")

if __name__ == "__main__":
    main()

```

INVESTIGACION

Aplicacion con SOAP Services

Se analizo el servicio publico "Calculator" de DNE Online. El WSDL define operaciones Add, Subtract, Multiply y Divide con tipos xsd:int, y expone bindings SOAP 1.1 y 1.2. Este contrato permite consumir el servicio desde herramientas como SoapUI, clientes JAX-WS y librerías como zeep.

Aspectos observados:

- Contrato formal con wsdl:service, wsdl:port y wsdl:binding.
- Mensajes SOAP con estructura XML fija y soapAction por operacion.
- Facil integracion para clientes multiplataforma mientras se respete el WSDL.

CUESTIONARIO

1. **JavaBeans y SOAP sobre JMS:** Si, un JavaBean puede formar parte de la implementacion, pero requiere un stack que soporte SOAP sobre JMS y configuracion de colas, bindings y metadatos.
2. **Mensajería bidireccional:** Se usa un canal de solicitud y una cola de respuesta (o JMSReplyTo) con correlationId para mapear respuestas. Soporta multiples clientes segun la capacidad del broker y la configuracion de consumidores concurrentes.
3. **Uso empresarial de SOAP:** Se mantiene por contratos WSDL, herramientas maduras y estandares WS- para seguridad, transacciones y confiabilidad, claves en entornos regulados.
4. **Impacto de XML:** Incrementa el tamaño de mensajes y el costo de parseo, elevando latencia y consumo de CPU/memoria en alta concurrencia. Se mitiga con compresion o streaming.
5. **Escenarios donde SOAP es mejor:** Integracion B2B, procesos con contratos estrictos, auditoria y requerimientos WS-Security/WS-ReliableMessaging, donde REST no cubre dichas garantias.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

CONCLUSION

Se implementaron servicios SOAP con JAX-WS y clientes en Java y Python, validando el flujo completo publicación-consumo. El uso de WSDL y contratos formales facilita la integración, aunque el consumo desde navegador presenta limitaciones prácticas por CORS y ausencia de soporte nativo.

REFERENCIAS Y BIBLIOGRAFIA

- [1] Tanenbaum, A. S. (2008). Sistemas distribuidos: principios y paradigmas. Mexico. Pearson Educacion.
- [2] Ceballos, F. J. (2006). Java 2, Curso de programación. Mexico: Alfaomega, Ra-Ma.
- [3] Deitel, H. M., & Deitel, P. J. (2004). Como programar en Java. Mexico: Pearson Educacion.
- [4] DNE Online Calculator WSDL. <https://www.dneonline.com/calculator.asmx?WSDL>
- [5] Zeep Documentation. <https://docs.python-zeep.org/>

ANEXOS

Servicio SOAP de Suma (Docente)

Servicio

```
package docente.calculadora;

import javax.ws.WebMethod;
import javax.ws.WebService;

@WebService(serviceName = "CalculadoraSOAPService", targetNamespace = "http://lab7.docente/")
public class CalculadoraSOAP {
    // START-SNIPPET,service
    @WebMethod
    public int sumar(int a, int b) {
        return a + b;
    }
    // END-SNIPPET
}
```

Publicación

```
package docente.calculadora;

import javax.xml.ws.Endpoint;

public class Publicador {
    // START-SNIPPET,publish
    public static void main(String[] args) {
        Endpoint.publish("http://localhost:8080/calculadora", new CalculadoraSOAP());
        System.out.println("Servicio SOAP activo");
    }
    // END-SNIPPET
}
```

Cliente

```
package docente.calculadora;

import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;

public class ClienteSOAP {
```


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

```
// START-SNIPPET,client
public static void main(String[] args) throws Exception {
    URL url = new URL("http://localhost:8080/calculadora?wsdl");
    QName qname = new QName("http://lab7.docente/", "CalculadoraSOAPService");
    Service service = Service.create(url, qname);
    CalculadoraSOAP calc = service.getPort(CalculadoraSOAP.class);
    System.out.println(calc.sumar(10, 20));
}
// END-SNIPPET
}
```

Ejercicio 1: Conversor de Temperatura

Interfaz

```
package l7.e1.conversor;

import javax.ws.WebMethod;
import javax.ws.WebService;

@WebService(targetNamespace = "http://l7.e1.conversor/")
public interface ConversorAPI {
    // START-SNIPPET,interface
    @WebMethod
    double cToF(double c);

    @WebMethod
    double fToC(double f);
    // END-SNIPPET
}
```

Implementacion

```
package l7.e1.conversor;

import javax.ws.WebService;

@WebService(
    endpointInterface = "l7.e1.conversor.ConversorAPI",
    serviceName = "ConversorSOAPService",
    targetNamespace = "http://l7.e1.conversor/"
)
public class ConversorSOAP implements ConversorAPI {
    // START-SNIPPET,impl
    @Override
    public double cToF(double c) {
        return (c * 9.0 / 5.0) + 32.0;
    }

    @Override
    public double fToC(double f) {
        return (f - 32.0) * 5.0 / 9.0;
    }
    // END-SNIPPET
}
```

Publicacion

```
package l7.e1.conversor;

import javax.xml.ws.Endpoint;

public class PublishService {
    // START-SNIPPET,publish
    public static void main(String[] args) {
        Endpoint.publish("http://localhost:8080/conversor", new ConversorSOAP());
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

```
// END-SNIPPET
}
```

Cliente

```
package l7.e1.conversor;

import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;

public class ConversorClient {
    // START-SNIPPET,client
    public static void main(String[] args) throws Exception {
        URL wsdlUrl = new URL("http://localhost:8080/conversor?wsdl");
        QName serviceName = new QName("http://l7.e1.conversor/", "ConversorSOAPService");
        Service service = Service.create(wsdlUrl, serviceName);
        ConversorAPI api = service.getPort(ConversorAPI.class);

        System.out.println("30C -> " + api.cToF(30));
        System.out.println("86F -> " + api.fToC(86));
    }
    // END-SNIPPET
}
```

Ejercicio 1: Ventas en Linea

Modelo Item

```
package l7.e1.store.model;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Item implements Serializable {
    private static final long serialVersionUID = 1L;

    // START-SNIPPET,model
    private String name;
    private int cantidad;
    private double costo;

    private static final List<Item> ITEMS = new ArrayList<>(Arrays.asList(
        new Item("Gaseosa", 15, 5.2),
        new Item("Galletas", 10, 1.6),
        new Item("Celular", 12, 900.0)));

    public Item() {
    }

    public Item(String name, int cantidad, double costo) {
        this.name = name;
        this.cantidad = cantidad;
        this.costo = costo;
    }

    public static List<Item> getItems() {
        return ITEMS;
    }

    public static boolean addItem(Item item) {
        if (item.name == null || item.name.isEmpty() || item.cantidad <= 0 || item.costo <= 0) {
            return false;
        }
        ITEMS.add(item);
        return true;
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

```

public static String buyItem(String name, int cantidad) {
    if (cantidad <= 0) {
        return "Invalid Quantity";
    }
    for (Item item : ITEMS) {
        if (item.name.equalsIgnoreCase(name)) {
            if (item.cantidad - cantidad < 0) {
                return "Not enough items";
            }
            item.cantidad -= cantidad;
            return "Producto: " + item.name + " Cantidad: " + cantidad
                + " Total: " + String.format(java.util.Locale.US, "%.2f", (cantidad * item.costo));
        }
    }
    return "Item no match";
}

public static boolean setItem(String name, int cantidad, double costo) {
    if (name == null || name.isEmpty() || cantidad <= 0 || costo <= 0) {
        return false;
    }
    for (Item item : ITEMS) {
        if (item.name.equalsIgnoreCase(name)) {
            item.cantidad = cantidad;
            item.costo = costo;
            return true;
        }
    }
    return false;
}

public static boolean deleteItem(String name) {
    if (name == null || name.isEmpty()) {
        return false;
    }
    for (Item item : ITEMS) {
        if (item.name.equalsIgnoreCase(name)) {
            item.cantidad = 0;
            return true;
        }
    }
    return false;
}

public String getNombre() {
    return name;
}

public void setNombre(String name) {
    this.name = name;
}

public int getCantidad() {
    return cantidad;
}

public void setCantidad(int cantidad) {
    this.cantidad = cantidad;
}

public double getCosto() {
    return costo;
}

public void setCosto(double costo) {
    this.costo = costo;
}

// END-SNIPPET

@Override
public String toString() {

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

```

    return "Item{name='" + name + "', cantidad=" + cantidad + ", costo=" + costo + "}";
}
}

```

Interfaz del Servicio

```

package l7.el.store.soap;

import java.util.List;
import javax.ws.WebMethod;
import javax.ws.WebService;
import l7.el.store.model.Item;

@WebService(targetNamespace = "http://l7.el.store/")
public interface SOAPI {
    // START-SNIPPET,interface
    @WebMethod
    List<Item> getItems();

    @WebMethod
    String buyItem(String name, int cantidad);

    @WebMethod
    boolean addItem(Item item);

    @WebMethod
    boolean setItem(String name, int cantidad, double precio);

    @WebMethod
    boolean deleteItem(String name);
    // END-SNIPPET
}

```

Implementación del Servicio

```

package l7.el.store.soap;

import java.util.List;
import javax.ws.WebService;
import l7.el.store.model.Item;

@WebService(
    endpointInterface = "l7.el.store.soap.SOAPI",
    serviceName = "StoreSOAPService",
    targetNamespace = "http://l7.el.store/"
)
public class SOAPImpl implements SOAPI {
    // START-SNIPPET,impl
    @Override
    public List<Item> getItems() {
        return Item.getItems();
    }

    @Override
    public String buyItem(String name, int cantidad) {
        return Item.buyItem(name, cantidad);
    }

    @Override
    public boolean addItem(Item item) {
        return Item.addItem(item);
    }

    @Override
    public boolean setItem(String name, int cantidad, double precio) {
        return Item.setItem(name, cantidad, precio);
    }

    @Override

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

```
public boolean deleteItem(String name) {
    return Item.deleteItem(name);
}
// END-SNIPPET
}
```

Publicacion del Servicio

```
package l7.e1.store.demo;

import javax.xml.ws.Endpoint;
import l7.e1.store.soap.SOAPIImpl;

public class PublishService {
    // START-SNIPPET,publish
    public static void main(String[] args) {
        Endpoint.publish("http://localhost:1516/WS/Store", new SOAPIImpl());
    }
    // END-SNIPPET
}
```

Cliente de Pruebas

```
package l7.e1.store.client;

import java.net.URL;
import java.util.Arrays;
import java.util.List;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;
import l7.e1.store.model.Item;
import l7.e1.store.soap.SOAPI;

public class StoreClient {
    // START-SNIPPET,client
    public static void main(String[] args) throws Exception {
        URL wsdlUrl = new URL("http://localhost:1516/WS/Store?wsdl");
        QName serviceName = new QName("http://l7.e1.store/", "StoreSOAPService");
        Service service = Service.create(wsdlUrl, serviceName);
        SOAPI store = service.getPort(SOAPI.class);

        List<Item> items = store.getItems();
        System.out.println("Lista de productos: " + Arrays.toString(items.toArray()));

        store.addItem(new Item("Chicle", 20, 1.4));
        System.out.println("Lista actualizada: " + Arrays.toString(store.getItems().toArray()));

        String recibo = store.buyItem("Galletas", 2);
        System.out.println("Recibo: " + recibo);

        store.setItem("Galletas", 18, 2.4);
        System.out.println("Lista final: " + Arrays.toString(store.getItems().toArray()));
    }
    // END-SNIPPET
}
```

Ejercicio 2: Cliente SOAP con Python

Script

```
# pyrefly: ignore [missing-import]
from zeep import Client

def main():
    wsdl_url = "http://localhost:1516/WS/Store?wsdl"
    print(f"Conectando al servicio SOAP en: {wsdl_url}...")
    try:
```



```
client = Client(wsdl_url)
print("¡Conexión establecida con éxito!\n")
except Exception as e:
    print(f"Error al conectar al servicio SOAP: {e}")
    return

while True:
    print("=====")
    print("          MENÚ DE OPERACIONES SOAP          ")
    print("=====")
    print("1. Listar todos los productos (getItems)")
    print("2. Comprar un producto (buyItem)")
    print("3. Registrar nuevo producto (addItem)")
    print("4. Actualizar stock y precio (setItem)")
    print("5. Eliminar producto (deleteItem)")
    print("6. Salir")
    print("=====")

    opcion = input("Seleccione una opción (1-6): ").strip()

    if opcion == "1":
        print("\n--- LISTA DE PRODUCTOS ---")
        try:
            items = client.service.getItems()
            if not items:
                print("No hay productos disponibles.")
            else:
                for item in items:
                    print(f"- Producto: {item.nombre:15} | Stock: {item.cantidad:5} | Costo: S/. {item.costo:.2f}")
        except Exception as e:
            print(f"Error al listar productos: {e}")
        print()

    elif opcion == "2":
        print("\n--- COMPRAR PRODUCTO ---")
        name = input("Nombre del producto: ").strip()
        if not name:
            print("Nombre inválido.")
            continue
        try:
            cantidad = int(input("Cantidad a comprar: "))
            resultado = client.service.buyItem(name, cantidad)
            print(f"Resultado: {resultado}")
        except ValueError:
            print("Por favor, ingrese una cantidad numérica válida.")
        except Exception as e:
            print(f"Error al realizar la compra: {e}")
        print()

    elif opcion == "3":
        print("\n--- REGISTRAR NUEVO PRODUCTO ---")
        name = input("Nombre del producto: ").strip()
        if not name:
            print("Nombre inválido.")
            continue
        try:
            cantidad = int(input("Cantidad inicial: "))
            costo = float(input("Costo/Precio: "))
            nuevo_item = {"nombre": name, "cantidad": cantidad, "costo": costo}
            bool = client.service.addItem(nuevo_item)
            if(bool):
                print(f"Producto '{name}' registrado exitosamente.")
            else:
                print(f"Hubo un problema y el Producto {name} no se pudo registrar")
        except ValueError:
            print("Por favor, ingrese valores numéricos válidos.")
        except Exception as e:
            print(f"Error al registrar producto: {e}")
        print()

    elif opcion == "4":
        print("\n--- ACTUALIZAR PRECIO Y STOCK ---")
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

```

name = input("Nombre del producto a actualizar: ").strip()
if not name:
    print("Nombre inválido.")
    continue
try:
    cantidad = int(input("Nueva cantidad (stock): "))
    costo = float(input("Nuevo costo/precio: "))
    bool = client.service.setItem(name, cantidad, costo)
    if(bool):
        print(f"Producto '{name}' actualizado exitosamente.")
    else:
        print(f'Hubo un problema y el Producto {name} no se pudo actualizar')
except ValueError:
    print("Por favor, ingrese valores numéricos válidos.")
except Exception as e:
    print(f"Error al actualizar producto: {e}")
print()

elif opcion == "5":
    print("\n--- ELIMINAR PRODUCTO (Baja lógica) ---")
    name = input("Nombre del producto a eliminar: ").strip()
    if not name:
        print("Nombre inválido.")
        continue
    try:
        bool = client.service.deleteItem(name)
        if(bool):
            print(f"Producto '{name}' eliminado lógicamente (stock establecido en 0).")
        else:
            print(f'Hubo un problema y el Producto {name} no se pudo eliminar')
    except Exception as e:
        print(f"Error al eliminar producto: {e}")
    print()

elif opcion == "6":
    print("\nSaliendo del cliente interactivo. ¡Adiós!")
    break
else:
    print("\nOpción no válida. Intente nuevamente.\n")

if __name__ == "__main__":
    main()

```